工业嵌入式 Bootloader 开发 Week 4 周报

项目：基于 QEMU AN500 的 Cortex M7 Bootloader　计划周次：Week 4

整理日期：2026-10-25。实验日期保留原日志；本报告按原计划第 4 周归纳已有固件校验、应用加载及跳转成果。老师反馈修订待审核，Week5 尚未推进。

一 本周任务与完成情况

本周复用 Week3 串口与存储接口，实现 CRC32 镜像完整性检查、独立应用加载、向量表验证与安全交接。最新阶段摘要 27 组 PASS，包括 5 组构建/CRC/交接/复制/诊断及 22 种镜像输入。旧记录的 23 组属于早期版本，不作为当前数量。

原计划第 4 周要求	本周成果	交付与边界
CRC32 完整性检查	头 CRC、payload CRC、复制后 CRC	crc32.c 与 image.c；不提供签名认证
从 Flash 到 RAM 加载	从主分区读取并复制至 0x00010000 SSRAM	32 字节镜像头不复制；来源为既有 NOR 模型
VTOR 栈与中断交接	清外部 IRQ 与 SysTick 状态，设置 VTOR，汇编切 MSP/BX	冷启动、特权线程、缓存及 MPU 未启用
测试应用和完整启动	独立应用自行初始化 data/bss 并验证 SVC	QEMU 不直接加载应用 ELF

Week2 板级/应用 UART 和 Week3 串口/存储为已有依赖，驱动开发与持久化研究不计入本周新增成果。Ymodem、主备升级、签名、看门狗和自动恢复属于 Week5–6。

二 固件校验 加载与交接

先确定镜像与地址约定

新增 32 字节小端镜像头，记录格式标识、格式版本、头长度、应用长度、固定加载地址、固件版本、内容 CRC32 和头 CRC32。Bootloader 先检查头和长度，再读取内容计算 CRC，随后检查栈顶、Thumb 入口与完整向量表。CRC 用于检测数据损坏，不提供签名认证或防回滚保证。

Bootloader 运行代码位于 0x00000000，应用代码加载到 0x00010000。二者的运行 RAM 分开：Bootloader 使用 0x20000000 至 0x20010000，应用使用 0x20010000 至 0x20200000；模拟存储位于 0x20300000 至 0x20400000。应用镜像从存储槽复制到运行区后，还要再次计算 CRC，防止加载过程中出现错误。

将处理器状态交给应用

跳转前关闭 UART、SysTick 和外部中断，清除挂起状态，设置应用 VTOR，再由汇编切换 MSP 和分支到应用复位入口。修改 MSP 后不再访问原 C 栈。应用自行完成数据初始化，确认交接状态后开启中断，并通过一次 SVC 调用验证应用向量表和异常返回。

```
BOOT: stage4 image validation
IMAGE: CRC32 PASS
LOAD: verified at 0x00010000
JUMP: application
APP: CRT PASS
APP: HANDOFF PASS
APP: SVC PASS
APP: RUNNING
```

以上内容来自本次真实 UART 日志。QEMU 只直接加载 Bootloader ELF，应用由 Bootloader 从镜像中复制并启动；因此应用输出能够用于验证实际加载与交接链路。

验证方法与定位经验

早期第四阶段共通过 23 项检查；当前扩展为 27 组，涵盖正常和最大长度镜像、空白镜像、错误格式、头和内容 CRC 损坏、超长内容、错误加载地址、非法 MSP 和入口、无效中断向量及加载后数据损坏。CRC 实现还在地址与未定义行为检查器下，与 Python zlib 的独立结果比较。

交接测试先让 CPU 实际执行中断屏蔽指令，再注入 NVIC、SysTick 和 PendSV 挂起状态。早期测试直接用 GDB 写 PRIMASK，但本机模型未按预期改变状态，导致注入的异常提前触发；改为执行指令并读回状态后，测试通过。这说明故障注入本身也需要验证，不能只看调试命令是否返回成功。

三 构建 演示与验证入口

在项目或交付包 codes 目录运行：

```
bash scripts/build-week4.sh
python3 scripts/run-week4.py
python3 scripts/verify-week4.py
```

build-week4.sh 会重新生成演示镜像，不应在其中存放需保留的实验数据。正常演示须出现 APP: RUNNING；GDB 交接证据还检查 app_status、CRT 与 SVC。宿主停止 QEMU 不是应用主动退出。

四 阶段执行与结果明细

W4LOG 为历史细化表证据 logs/week4-20261024-173015-915034。镜像输入的最新 22 项及生产回归使用 logs/week4-20261025-015917-068252。按用例编号关联步骤、输入、预期、实际和日志，避免把不同日期的数量混为当前结果。

执行表

日志根目录：W4LOG = logs/week4-20261024-173015-915034（相对 codes/）。表内路径由该根目录和文件名组成。

固定编号	测试方式	操作步骤	具体输入
W4-BUILD	构建/地址检查	1. 新目录构建 Bootloader 和独立应用；2. objcopy 生成 application.bin；3. 检查两 ELF 反汇编、符号及栈/向量地址。	独立 Boot/App ELF；固定栈、VTOR 和链接段
W4-CRC	独立 zlib oracle 及 sanitizer	1. 编译真实 crc32.c 和 test_crc32.c，启用 sanitizer；2. 用固定随机种子生成 8 种长度输入；3. 逐次运行；4. 将结果与独立 zlib.crc32 比较。	长度 0,1,9,255,256,257,4096,458720，种子 20261022；标准串 123456789
W4-HANDOFF	GDB 真实 CPSID 后注入	1. 在 boot_handoff 停下并实际执行 CPSID；2. 读回 PRIMASK=1 后注入挂起状态；3. 在应用 Reset 入口检查寄存器；4. 污染应用 data/bss 和设置哨兵；5. 运行到 app_idle，检查 CRT、交接、SVC 状态。	NVIC IRQ7、SysTick、PendSV 挂起；应用 data/bss 污染
W4-COPY	GDB 复制后翻转	1. 正常镜像启动；2. 在 boot_verify_loaded 入口断点暂停；3. 翻转运行地址 0x100c8 一位；4. 继续到 boot_idle；5. 检查 boot_status 和完整串口字节。	运行地址 0x100c8 一个字节异或 1
W4-DIAG	模型日志检查	1. 遍历镜像场景及 GDB 场景的 qemu.log；2. 排除主动终止消息；3. 断言没有其他 guest-error。	所有正常/拒绝场景 qemu.log

结果与证据表

固定编号	预期输出	实际输出	判定	日志路径（根目录见上）
W4-BUILD	无 BKPT，地址符合	断言通过	PASS	W4LOG/build.txt；W4LOG/summary.json
W4-CRC	CRC 与 zlib 逐一相等；标准串 cbf43926	全部相等，无 sanitizer 错误	PASS	W4LOG/summary.json；W4LOG/summary.json
W4-HANDOFF	入口 MSP=20200000、VTOR=10000、PRIMASK=1；清中断；应用初始化恢复且 SVC=1	APP MSP VTOR masks NVIC SysTick CRT sentinel SVC: PASS	PASS	W4LOG/dirty-state-handoff-and-app-crt.txt；W4LOG/summary.json

固定编号	预期输出	实际输出	判定	日志路径（根目录见上）
W4-COPY	状态 8，REJECT 00000008，无应用日志	boot_status=8；输出符合	PASS	W4LOG/reject-corrupted-loaded-copy.txt；W4LOG/summary.json
W4-DIAG	无 guest error	检查通过	PASS	W4LOG/summary.json；W4LOG/summary.json

头起点为镜像偏移 0x10000，payload 起点为 0x10020。22 种镜像输入的 SHA256、完整头和 48 项向量十六进制值在 summary.json/image_cases 中。向量负向输入修复 CRC 后再测试，避免 CRC 错误遮蔽向量检测。CRC 检查不认证来源；备份槽尚不用于启动。

镜像输入逐项对照

本次按老师要求将原有 18 种输入拆为 18 行主表，已增加的 4 种边界输入列为补充表，保留 IMG 编号。表格依据已通过的 logs/week4-20261025-015917-068252，沿用实际执行结果，不将文档编辑算作重新测试。构造输入经重新还原，22 份镜像 SHA256 均与原始 summary.json 一致；具体数值已与头、向量、宿主输出及 QEMU UART 逐项核对。

偏移均以整个 1 MiB 镜像文件起点为 0；H = 0x00010000 为镜像头起点，P = H + 0x20 = 0x00010020 为 payload/向量表起点。除注明“一个字节”及全文件/填充区外，修改均为小端 uint32；区间均左闭右开。每行从同一有效基准独立构造，不将前一行修改带到下一行。基准 payload 为 972 字节，固件版本 1。

“头重算”指对[H,H+0x1C)计算 CRC32 并写至 0x0001001C；“内容 CRC”对声明的 payload 计算并写至 0x00010018。向量修改先重算内容 CRC 再重算头 CRC，以免较早的 CRC 错误遮蔽目标向量错误。头字段负向输入只重算头 CRC；头 CRC 错误及内容损坏用例保留旧 CRC。

IMGLOG = logs/week4-20261025-015917-068252（相对 codes/）。表内 N 为宿主 image_validate 的 IMAGE_RESULT，不是宿主进程退出码；预期拒绝时测试程序断言匹配后退出 0。Q 为 QEMU UART 实际结果，“拒绝 0xNN”对应完整 REJECT 十六进制行且没有 APP: RUNNING；成功须出现 APP: RUNNING。每行日志均在 IMGLOG 根目录下，前者为宿主输出，后者为 UART。详细 CRC 数值、实际 SIZE、输入 SHA256 和完整 UART 另存 codes/docs/week4-image-inputs.json。

错误码对应：0=接受；2=格式；3=头 CRC；4=长度；5=加载地址；6=内容 CRC；7=向量。复制后运行区损坏对应 8，属于 W4-COPY 的另一类 GDB 故障注入，不计入这 18+4 种输入。CRC 只检测损坏，不证明固件来源可信。

原有 18 种镜像输入

编号与场景	修改偏移	具体数值	CRC 处理	预期错误码	实际结果与判定	日志（相对 IMGLOG）
IMG-01 valid	无修改	有效基准；version=1；payload_size=972	打包时计算两 CRC	0 (IMAGE_OK)	N=0；Q=APP: RUNNING；PASS	valid-native.txt；valid/uart.log
IMG-02 blank	全文件 [0x00000000,0x00100000)	每字节=0xFF；1 MiB	均不计算	0x02	N=2；Q=拒绝 0x02；PASS	blank-native.txt；blank/uart.log
IMG-03 bad-magic	H+0x00 = 0x00010000	magic：0x4D49374D → 0x00000000	头重算；内容保留	0x02	N=2；Q=拒绝 0x02；PASS	bad-magic-native.txt；bad-magic/uart.log
IMG-04 unsupported-format	H+0x04 = 0x00010004	format：1 → 2	头重算；内容保留	0x02	N=2；Q=拒绝 0x02；PASS	unsupported-format-native.txt；unsupported-format/uart.log
IMG-05 bad-header-size	H+0x08 = 0x00010008	header_size：32 → 64	头重算；内容保留	0x02	N=2；Q=拒绝 0x02；PASS	bad-header-size-native.txt；bad-header-size/uart.log
IMG-06 bad-header-crc	H+0x14 = 0x00010014	firmware_version：1 → 8；保留旧头 CRC	均不重算	0x03	N=3；Q=拒绝 0x03；PASS	bad-header-crc-native.txt；bad-header-crc/uart.log
IMG-07 empty-payload	H+0x0C = 0x0001000C	payload_size：972 → 0	头重算；内容保留	0x04	N=4；Q=拒绝 0x04；PASS	empty-payload-native.txt；empty-payload/uart.log
IMG-08 oversize-payload	H+0x0C = 0x0001000C	payload_size：972 → 0xFFFFFFFF	头重算；内容保留	0x04	N=4；Q=拒绝 0x04；PASS	oversize-payload-native.txt；oversize-payload/uart.log
IMG-09 wrong-load-address	H+0x10 = 0x00010010	load_address：0x00010000 → 0x20000000	头重算；内容保留	0x05	N=5；Q=拒绝 0x05；PASS	wrong-load-address-native.txt；wrong-load-address/uart.log
IMG-10 payload-corruption	P+0xC8 = 0x000100E8	一个字节：0x91 → 0x90（异或 1）	均不重算	0x06	N=6；Q=拒绝 0x06；PASS	payload-corruption-native.txt；payload-corruption/uart.log
IMG-11 unaligned-msp	P+0x00 = 0x00010020	vector[0]：0x20200000 → 0x20200004	先内容 CRC，再头 CRC	0x07	N=7；Q=拒绝 0x07；PASS	unaligned-msp-native.txt；unaligned-msp/uart.log

编号与场景	修改偏移	具体数值	CRC 处理	预期错误码	实际结果与判定	日志（相对 IMGLOG）
IMG-12 boot-ram-msp	P+0x00 = 0x00010020	vector[0]：0x20200000 → 0x20010000	先内容 CRC，再头 CRC	0x07	N=7；Q=拒绝 0x07；PASS	boot-ram-msp-native.txt；boot-ram-msp/uart.log
IMG-13 even-reset	P+0x04 = 0x00010024	vector[1]：0x000100C1 → 0x000100C0	先内容 CRC，再头 CRC	0x07	N=7；Q=拒绝 0x07；PASS	even-reset-native.txt；even-reset/uart.log
IMG-14 outside-reset	P+0x04 = 0x00010024	vector[1]：0x000100C1 → 0x00080001	先内容 CRC，再头 CRC	0x07	N=7；Q=拒绝 0x07；PASS	outside-reset-native.txt；outside-reset/uart.log
IMG-15 vector-table-reset	P+0x04 = 0x00010024	vector[1]：0x000100C1 → 0x00010001	先内容 CRC，再头 CRC	0x07	N=7；Q=拒绝 0x07；PASS	vector-table-reset-native.txt；vector-table-reset/uart.log
IMG-16 bad-irq-vector	P+0x40 = 0x00010060	vector[16]：0x00010105 → 0x00000000	先内容 CRC，再头 CRC	0x07	N=7；Q=拒绝 0x07；PASS	bad-irq-vector-native.txt；bad-irq-vector/uart.log
IMG-17 reserved-vector	P+0x1C = 0x0001003C	vector[7]：0x00000000 → 0x000100C1	先内容 CRC，再头 CRC	0x07	N=7；Q=拒绝 0x07；PASS	reserved-vector-native.txt；reserved-vector/uart.log
IMG-22 maximum-payload	H+0x0C；文件 [0x000103EC,0x00080000)	长度 972 → 458720 (0x6FFE0)；扩展部分填 0xFF	重新打包计算两 CRC	0 (IMAGE_OK)	N=0；Q=APP: RUNNING；PASS	maximum-payload-native.txt；maximum-payload/uart.log

新增 4 种边界输入

编号与场景	修改偏移	具体数值	CRC 处理	预期错误码	实际结果与判定	日志（相对 IMGLOG）
IMG-18 msp-above-top	P+0x00 = 0x00010020	vector[0]：0x20200000 → 0x20200008	先内容 CRC，再头 CRC	0x07	N=7；Q=拒绝 0x07；PASS	msp-above-top-native.txt；msp-above-top/uart.log
IMG-19 msp-below-base	P+0x00 = 0x00010020	vector[0]：0x20200000 → 0x2000FFF8	先内容 CRC，再头 CRC	0x07	N=7；Q=拒绝 0x07；PASS	msp-below-base-native.txt；msp-below-base/uart.log

编号与场景	修改偏移	具体数值	CRC 处理	预期错误码	实际结果与判定	日志（相对 IMGLOG）
IMG-20 reset-at-end	P+0x04 = 0x00010024	vector[1]：0x000100C1 → 0x000103CD；(0x10000+972)按位或 1	先内容 CRC，再头 CRC	0x07	N=7；Q=拒绝 0x07；PASS	reset-at-end-native.txt；reset-at-end/uart.log
IMG-21 irq-before-code	P+0x40 = 0x00010060	vector[16]：0x00010105 → 0x0000FFFF	先内容 CRC，再头 CRC	0x07	N=7；Q=拒绝 0x07；PASS	irq-before-code-native.txt；irq-before-code/uart.log

五 Week4 一键入口测试

证据根目录 CLI34LOG = logs/cli-week34-20261024-234802。跨周专项 22 项中，本周列出 Week4 的 11 项。测试在独立副本中执行真实构建/运行命令，检查退出码、QEMU 回收、临时目录和文件锁；5 秒内部启动/UART 超时与 SIGTERM 后最多 3 秒再 SIGKILL 的清理路径均有日志，20 秒外层看门狗未触发。构建依赖失败保留 42，运行失败返回 1。

Week 4 执行表

固定编号	实际命令	操作步骤	具体输入
CLI34-W4-MISSING	python3 scripts/run-week4.py	复制工程时排除 build；直接运行默认命令	ELF 和镜像均不存在
CLI34-W4-BUILD	bash scripts/build-week4.sh	确认不存在 build；执行交付构建命令	原始工程，无构建缓存
CLI34-W4-RUN	python3 scripts/run-week4.py	构建后执行默认运行命令；检查输出及清理	默认有效镜像
CLI34-W4-MISSING-ELF	python3 scripts/run-week4.py	保留镜像；仅移走 ELF；直接运行	ELF 不存在
CLI34-W4-MISSING-IMAGE	python3 scripts/run-week4.py	保留 ELF；移走镜像；直接运行	默认镜像不存在
CLI34-W4-QEMU-FAIL	python3 scripts/run-week4.py	只在副本替换 QEMU 启动依赖；执行原运行命令	子命令返回 42
CLI34-W4-START-TIMEOUT	python3 scripts/run-week4.py	只在副本替换 QEMU 启动依赖；执行原运行命令	子进程不建 socket 且忽略 SIGTERM
CLI34-W4-UART-TIMEOUT	python3 scripts/run-week4.py	仅将副本 ELF 换成无限循环固件；运行真实 QEMU	有效 ELF，无 UART 输出
CLI34-W4-FAIL-CONFIGURE	bash scripts/build-week4.sh	保留旧产物；注入 cmake 失败；运行交付构建命令	cmake configure 返回 42
CLI34-W4-FAIL-BUILD	bash scripts/build-week4.sh	保留旧产物；注入 cmake 失败；运行交付构建命令	cmake build 返回 42
CLI34-W4-RECOVERY	python3 scripts/run-week4.py	恢复依赖和 ELF 后再次运行，检验无残留锁/进程干扰	原有效 ELF 和镜像；Week3 输入 quit

Week 4 结果与证据

固定编号	预期输出及退出码	实际输出及清理	判定	日志路径
CLI34-W4-MISSING	退出 1；不启动 QEMU	exit=1；0.037 秒；外层看门狗未触发。未启动 QEMU。image missing	PASS	CLI34LOG/CLI34-W4-MISSING.log
CLI34-W4-BUILD	退出 0，生成该阶段 ELF 及所需镜像	exit=0；0.413 秒；外层看门狗未触发。未启动 QEMU。构建产物存在	PASS	CLI34LOG/CLI34-W4-BUILD.log
CLI34-W4-RUN	退出 0；固件成功标记；QEMU 退出、临时目录删除、文件锁释放	exit=0；0.226 秒；外层看门狗未触发。子进程已回收；临时目录已删除；文件锁已释放。APP: RUNNING	PASS	CLI34LOG/CLI34-W4-RUN.log
CLI34-W4-MISSING-ELF	退出 1；ELF not found；不启动 QEMU	exit=1；0.033 秒；外层看门狗未触发。未启动 QEMU。ELF not found	PASS	CLI34LOG/CLI34-W4-MISSING-ELF.log
CLI34-W4-MISSING-IMAGE	退出 1；image missing；不启动 QEMU	exit=1；0.033 秒；外层看门狗未触发。未启动 QEMU。image missing	PASS	CLI34LOG/CLI34-W4-MISSING-IMAGE.log
CLI34-W4-QEMU-FAIL	入口退出 1；无成功标记；清理子进程、临时目录和文件锁	exit=1；0.06 秒；外层看门狗未触发。子进程已回收；临时目录已删除；文件锁已释放。QEMU exited before opening sockets	PASS	CLI34LOG/CLI34-W4-QEMU-FAIL.log
CLI34-W4-START-TIMEOUT	入口退出 1；无成功标记；清理子进程、临时目录和文件锁	exit=1；8.052 秒；外层看门狗未触发。子进程已回收；临时目录已删除；文件锁已释放。QEMU socket startup timeout	PASS	CLI34LOG/CLI34-W4-START-TIMEOUT.log
CLI34-W4-UART-TIMEOUT	约 5 秒内部 UART 超时；入口退出 1；QEMU 和资源清理完成	exit=1；5.086 秒；外层看门狗未触发。子进程已回收；临时目录已删除；文件锁已释放。ERROR: timed out	PASS	CLI34LOG/CLI34-W4-UART-TIMEOUT.log
CLI34-W4-FAIL-CONFIGURE	退出 42；不被旧产物掩盖；不启动 QEMU	exit=42；0.025 秒；外层看门狗未触发。未启动 QEMU。CMake 配置失败	PASS	CLI34LOG/CLI34-W4-FAIL-CONFIGURE.log
CLI34-W4-FAIL-BUILD	退出 42；不被旧产物掩盖；不启动 QEMU	exit=42；0.048 秒；外层看门狗未触发。未启动 QEMU。CMake 构建失败	PASS	CLI34LOG/CLI34-W4-FAIL-BUILD.log
CLI34-W4-RECOVERY	退出 0 并完成清理	exit=0；0.087 秒；外层看门狗未触发。子进程已回收；临时目录已删除；文件锁已释放。APP: RUNNING	PASS	CLI34LOG/CLI34-W4-RECOVERY.log

六 复用驱动的验证与 TX 错误传播

Week4 复用 Week3 驱动，不重复计入 UART 开发成果。波特率沿用 25000000/115200，商 217、余数 1600，小于四舍五入阈值 57600；分频 217，理论 115207.3732718894 baud，误差+0.006400409626%。BAUD-W4 在全 FF 镜像拒绝路径读回 Bootloader BAUDDIV=0xD9、CTRL=0x0B、pclk=25000000；证据 logs/uart-baud-20261024-235338/week4-registers.txt 及 summary.json。它不是应用 UART 重初始化后的读回，也不代表物理波形测量。

PRIMASK 接口契约和两种初值恢复测试作为 Week3 已有依赖，详见 Week3 周报第七节和 codes/docs/irq-contract-validation.md。本周交接另由 W4-HANDOFF 验证掩码、VTOR、栈和中断清理。

TXLOG = logs/tx-full-20261025-015904。本周引用跨周专项的以下路径，不将整套 7 组再次计为独立 Week4 新增测试。Bootloader 用 Week3 驱动，应用用 Week2 驱动；两版驱动返回值与计数的 10 项断言分别保留在 week3-driver.txt、week2-driver.txt。

固定编号	操作步骤与具体输入	预期结果	实际结果
TX-PRODUCTION	重新构建 3 个正常目标；nm 检查符号	无注入变量	三个 ELF 均不含 uart_tx_test_*；PASS
TX-W4-PRINT	有效镜像，持续 FULL=1；运行实际 Week4 main；断在 boot_idle	初次输出失败；boot_status=0x101；不跳转应用	0x101；读取 1000000 次；DATA=0x55；PASS
TX-W4-FLUSH	有效镜像；FULL=0 完成校验和输出；在 main 的最终 flush 前改为 FULL=1	最终 flush 失败，boot_status=0x101；停在 boot_idle	0x101；该次 flush 新增读取 1000000 次；PASS
TX-W4-APP	正常 Bootloader 加载有效的注入应用；应用 FULL=1；断在 app_idle	交接正常；应用 print 失败；app_status=4；不进入运行成功状态	handoff_ok=1；app_status=4；读取 1000000 次；DATA=0x55；PASS

TX-PRODUCTION 证据为 TXLOG/bootloader-week3-symbols.txt、bootloader-week4-symbols.txt、application-symbols.txt。其他 4 组各对应 TXLOG/编号.txt、编号.gdb、编号-uart.log。例如 TX-W4-FLUSH 对应 TXLOG/TX-W4-FLUSH.txt。

四条错误路径 GDB 断言均退出 0，均没有 PONG 或 APP: RUNNING。Week4 应用用例允许 Bootloader 先输出自身的 CRC/LOAD 成功信息；最终是否运行成功须同时检查 app_status 和 APP: RUNNING，不能把之前的中间成功行当作应用成功。GDB 取证后由测试脚本终止并回收 QEMU，不能将这个宿主清理退出码当作固件成功退出。

每次 flush 最多读 1000000 次，失败上报到 Bootloader 0x101 或应用状态 4；预算不是毫秒。注入验证实际控制流，不等同于 QEMU 字符后端物理拥塞或真机超时测量。成功必须结合 APP: RUNNING 与应用状态，不能用先前的 CRC/LOAD 中间成功行替代。

七 向量边界 状态字典与交付清单

以下补充对应老师对镜像、范围、状态图、REJECT 及 manifest 的反馈，归属计划 Week4。321 项宿主边界断言与 QEMU/GDB 路径、41 组 manifest 校验各有独立证据；断言、检查分组与镜像输入数不直接相加。

向量合法范围与边界覆盖

本工程固定 AN500、48 项向量和 192 字节向量表。范围规则来自 src/week4/image.c 的 vectors_ok，image_validate 与 boot_verify_loaded 共同调用；宿主打包器 scripts/image_format.py 实施相同的输入规则。下表是本工程镜像契约，不将其扩展为所有 Cortex-M 芯片的统一地址规则。

检查对象	精确规则	边界含义	验证与证据
向量表位置	payload 复制到 0x00010000；向量表 [0x10000,0x100C0)，共 48 项	load 必须为固定地址；表本身按 256 字节对齐	W4-BUILD、W4-HANDOFF；链接与 GDB 证据
初始 MSP vector[0]	0x20010000 < MSP <= 0x20200000；MSP 模 8=0	下界不接受；上界接受；保留 Bootloader RAM	VEC-001..015；MSP-AT-BASE
Reset vector[1]	entry 位 0 必须 1；addr=entry 按位与 0xFFFFFFFE	Thumb 位与地址分别检查，不按 4 字节对齐处理	Reset 索引 7 个边界；RESET-BELOW
所有已实现异常及 IRQ 入口	0x100C0 <= addr < 0x10000+payload_size，且 entry 位 0=1	代码下界接受；上界不接受；零入口拒绝	42 个必填索引各 7 项；VEC-MATRIX
核心保留项	索引 7、8、9、10、13 必须为 0	每一项单独验证 0 可接受、非 0 拒绝	5 项各 2 个断言
外部 IRQ	索引 16..47 对应 IRQ0..31，每项必须为非零合法 Thumb 入口	未使用 IRQ 也要配置合法默认处理函数	全部 32 项都测，不仅测 IRQ0
复制后的复核	先 CRC，再用同一规则检查 MSP 及全部入口	先修正测试 CRC 才能直接触达向量拒绝分支	LOAD-VECTOR-MSP、LOAD-VECTOR-IRQ31

新证据 VLOG = logs/vector-bounds-20261025-021521（相对 codes/）。python3 scripts/verify-vector-bounds.py 在干净构建中执行 321 个宿主断言（ASan/UBSan），再做 4 个 CPU 镜像拒绝、2 个复制后向量复核及 2 个状态路径验证。完整 321 项的输入、预期、实际、文件行号见 codes/docs/vector-boundary-cases.json；原始输出为 VLOG/matrix.txt。

覆盖类别	输入及预期	本轮结果
MSP 上下界	0、base-8、base、base+8、top-8、top、top+8、0xFFFFFFF8	只有 base+8、top-8、top 接受；8 项 PASS
MSP 对齐	base+8 再分别加 1..7	7 种余数全部返回 7；PASS
每个必填入口	最低合法 Thumb 地址、低于代码区、最高内部地址、等于末端、超过末端、清 Thumb 位、零	42×7=294 项 PASS
每个保留项	零与 0x100C1	5×2=10 项 PASS
表长边界	payload=191、192	191 返回长度错误 4；192 无代码可容纳合法入口，返回向量错误 7；2 项 PASS
CPU 拒绝	MSP 恰为 RAM 下界、保留项 13 非零、IRQ31 恰到末端、Reset 低于代码区	各输出 REJECT: 0x00000007；4 项 PASS
加载后复核	运行区 MSP 改为下界、IRQ31 入口改零；更新 boot_image.crc 以越过 CRC 检查	两项均在 boot_verify_loaded 返回 7；PASS

对于基准 972 字节 payload，代码起始地址范围为[0x100C0,0x103CC)；最低合法 entry 为 0x100C1，最高合法 entry 为 0x103CB，entry=0x103CD 解码为末端，拒绝。接受边界值的宿主用例只验证地址判定，没有把修改成任意内部地址的镜像启动执行。当前规则只检查入口起始地址、Thumb 位及范围，不证明该地址上的完整指令或函数语义有效；实际应用运行证据仍来自未修改的合法应用。最低可执行镜像还须包含代码，不能把 192 字节长度字段检查通过解释为整个镜像通过。

完整状态转移图



Bootloader 异常：记录 exception_number，boot_status = 0x100 → boot_idle；通常没有 REJECT 行。
boot_status = 0 在跳转前再次写入，单看 0 不能区分“尚未初始化状态”和“已交接”。
UART 在 REJECT 输出途中失败：可能仅输出部分文本，最终 boot_status 被改写为 0x101。

完整启动状态及失败去向

图中所有 Bootloader 失败最终都进入 boot_idle 循环，应用自己的成功或失败最终进入 app_idle。当前没有备份槽切换、重试下载或自动恢复分支。完整可编辑图源为 codes/docs/images/boot-state-complete.mmd，渲染脚本为 codes/scripts/render-boot-state.py。图中“通过”表示继续下一步，不表示最终应用运行成功。

boot_status 不是进度枚举：它在 BSS 初始化时为 0，成功交接前也会被写为 0；中间校验阶段没有逐状态写入。判断应用成功须结合 APP: RUNNING、app_status=1 及交接/CRT/SVC 检查。应用出错时 boot_status 可能仍为 0，应读取 app_status 和 app_exception。

REJECT 输出与状态字典

src/week4/main.c 的 reject(result)先把 result 写入 boot_status，再依次输出前缀、十六进制码、CRLF 和提示符，最后进入 boot_idle。精确格式如下，其中 h 为小写十六进制数字；引号表示字符串边界，不是发送字节：

"REJECT: 0xhhhhhhhh\r\n> "

前缀中的空格、0x、恰好 8 位十六进制和末尾提示符后的空格都是输出的一部分。\r\n 表示 CR/LF 两个字节，不是发送反斜线字符；提示符之后没有额外换行。唯一字段是无符号 32 位错误码，来源是 image_result，不含地址、时间、序号，也不是宿主进程退出码。

boot_status 值	名称与触发条件	REJECT 行为	最终停留及证据
0	BSS 初值，或完成最终 flush 后准备交接	不输出 REJECT；单值 0 不证明应用成功	初期位置不定；交接后应用 app_idle；W4-HANDOFF
1	IMAGE_IO：设备未绑定、读取失败等	REJECT: 0x00000001	boot_idle；VLOG/STATUS-IO.txt 及-uart.log
2	IMAGE_FORMAT_ERROR：magic、format、header_size 不符	REJECT: 0x00000002	boot_idle；IMG-02..05
3	IMAGE_HEADER_CRC：前 28 字节 CRC 不匹配	REJECT: 0x00000003	boot_idle；IMG-06
4	IMAGE_LENGTH：长度小于表、超过槽/执行区，或 load/verify 重复边界无效	REJECT: 0x00000004	boot_idle；IMG-07/08；load 参数分支按源码映射
5	IMAGE_ADDRESS：load 不等于 0x10000	REJECT: 0x00000005	boot_idle；IMG-09
6	IMAGE_PAYLOAD_CRC：存储 payload CRC 不匹配	REJECT: 0x00000006	boot_idle；IMG-10
7	IMAGE_VECTOR：存储或运行区向量非法	REJECT: 0x00000007	boot_idle；向量矩阵及 LOAD-VECTOR 两项
8	IMAGE_COPY_CRC：运行区 CRC 与已校验 info.crc 不符	REJECT: 0x00000008	boot_idle；W4-COPY

boot_status 值	名称与触发条件	REJECT 行为	最终停留及证据
0x100	CPU 异常：记录 exception_number=IPSR，再写状态	异常路径不调用 reject，无 REJECT 保证	boot_idle；VLOG/STATUS-HARDFAULT.txt 读回 exception_number=3
0x101	UART 初始化、write 或最终 flush 失败	UART 不可用，不保证错误文本或完整 REJECT	boot_idle；TX-W4-PRINT、TX-W4-FLUSH

VLOG = logs/vector-bounds-20261025-021521（相对 codes/）；IMG 编号对应第四阶段输入表的宿主和 UART 日志，TX 编号对应 TX 专项报告。状态 1 由 GDB 在 image_validate 入口将已绑定设备的 memory 指针置零触发实际 flash_read 失败；0x100 由无效取指触发真实 HardFault 验证。它们分别是数据读取错误注入和 CPU 异常测试，不是篡改 boot_status 伪造结果。各 load 参数失败的所有子条件尚未逐一做 GDB 注入，应以表中具体验证范围为准。

应用使用独立状态：app_status=0 为初值，1 为 CRT/交接/SVC 通过后的运行状态，2 为 CRT、交接或 SVC 检查失败，3 为应用异常（记录 app_exception），4 为 UART 失败。最终均在 app_idle 循环。app_status=4 已由 TX-W4-APP 验证；不能将应用状态值套用为 Bootloader 的 image_result。

格式错误示例（只展示拒绝行及提示符）：

"REJECT: 0x00000002\r\n> "

复制损坏示例的完整 UART 为：

BOOT: stage4 image validation\r\n
IMAGE: CRC32 PASS\r\n
"REJECT: 0x00000008\r\n> "

UART 在 reject 输出途中失败时，可能留下不完整前缀，boot_status 最终被 print 改写为 0x101；这时以 GDB 状态取证，不要求再靠同一 UART 输出错误行。IRQ 唤醒后 idle 函数仍循环，不自动恢复启动流程。

独立 manifest schema 与示例

交付 manifest 是包根目录的 delivery-manifest.json，用于记录列出的交付文件路径、字节数和 SHA256，帮助老师检查复制完整性及复核材料。它不进入 Bootloader 解析流程；固件仍使用 32 字节 M7IM 二进制头。manifest 不是签名或老师批准证明，approval_status 固定为 pending-teacher-review。

独立 schema 路径为 codes/schemas/delivery-manifest.schema.json，采用 JSON Schema Draft 2020-12；实现为 codes/scripts/delivery_manifest.py。校验器实现本 schema 使用的 type、required、additionalProperties、const、enum、pattern、minItems、minimum 及 date-time 检查，不作为任意 schema 的通用执行器。

字段	约束	本轮检验
根对象	6 字段全部必填；不允许额外字段	各字段缺失及 UNKNOWN 均拒绝

字段	约束	本轮检验
schema_version	常量整数 1，布尔 true 不作 1 接受	VERSION、BOOLEAN-VERSION
plan_group	week1-2、week3、week4；week3-4 兼容历史包	GROUP 拒绝 week5
generated_at	ISO 日期时间，必须+08:00；秒必填，可有小数；日期必须真实有效	DATE、DATE-TEXT、ZONE
image_format	常量 M7IM-v1	FORMAT
approval_status	常量 pending-teacher-review	APPROVAL 拒绝 approved
files	非空数组；每项只允许 path、size、sha256，均必填	FILES-TYPE、EMPTY、FILE-MISSING 等
files[].path	相对 POSIX 路径；无绝对路径、盘符、反斜线、控制字符、.或..分段、双斜线；规范化后不变	TRAVERSAL、ABSOLUTE、DOT 等
files[].size	非负整数，拒绝布尔和小数	SIZE、BOOL-SIZE、FLOAT-SIZE
files[].sha256	64 个小写十六进制字符	HASH、HASH-UPPER

规范化路径、重复路径拒绝、真实文件存在性、长度/哈希、符号链接越界属于校验器的附加语义检查。基本字段形状由 schema 约束；schema 本身不读取文件。生成器扫描目录中的文件，排除 delivery-manifest.json 自身以避免自引用；校验器核对清单列出的文件，不声称对未列出的额外文件做完整性认证。

有效示例完整 JSON 如下。配套 README.txt 字节存放在 codes/schemas/examples/，其长度和 SHA256 与清单一致。


```
{
"schema_version": 1,
"plan_group": "week3-4",
"generated_at": "2026-10-25T02:16:04.132229+08:00",
"files": [
{
"path": "README.txt",
"size": 25,
"sha256": "798fea15a540e232f4ae06c1888d0467da504195abb547083ebfec97ceb955fb"
}
],
"image_format": "M7IM-v1",
"approval_status": "pending-teacher-review"
}
```


无效示例：缺少必填 files 字段，应抛出 ValueError，不能作为空清单接受。

{
"schema_version": 1,
"plan_group": "week3-4",
"generated_at": "2026-10-25T02:16:04.132229+08:00",
"image_format": "M7IM-v1",
"approval_status": "pending-teacher-review"
}

完整的另外两份无效 JSON 为 schemas/examples/invalid-traversal.json 和 invalid-hash.json，分别把 path 设为../escape 和 sha256 设为 xyz。有效与三个无效示例均在本轮脚本中生成并验证；校验成功 0 退出，字段/文件失败抛异常，命令非零退出。

复验：在 codes/执行 python3 scripts/verify-manifest.py。本轮 41 项全部通过，证据 MLOG = logs/manifest-20261025-021604（相对 codes/），每项完整输入 JSON 和预期/实际异常见 summary.json 及对应 JSON 文件，包括有效实例、6 个必填字段、各标量/数组类型、日期、大小、哈希、路径、重复、缺失文件与符号链接越界。

生成正式交付清单：python3 scripts/delivery_manifest.py 包目录 --group week4；再次核验：python3 scripts/delivery_manifest.py 包目录。打包后还校验 ZIP 完整性。清单能检测非故意复制错误，但 SHA256 清单可被一同修改，不具备签名认证能力。

八 本周交付与后续安排

正式材料位于 reports/week4/，本周交付清单见 delivery-index.md 和 planning/week3-week4-delivery-index.md。交付包附 Week4 源码、测试应用、必要 Week2/3 依赖、测试细节及原始证据。manifest 用于文件完整性核查，不是固件签名或老师验收证明。

收到老师对当前修改的确认后，再按第 5 周计划开展 Ymodem、主备升级与升级状态管理。当前不因已有代码可运行而跳过反馈审核，也不把现有工程称为八周最终工业级交付。

2026-10-25 交付枚举扩展：单周包分别使用 plan_group=week3 或 week4；保留 week3-4 用于历史兼容。上文 02:16 的测试实例与 41 组记录保留原执行版本，新增单周枚举另以实际交付包生成和校验验证。